LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

Android Basics With Compose

Oleh:

Hamka Arifani NIM. 2410817110006

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MARET 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile . Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Hamka Arifani
NIM : 2410817110006

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
Soal Praktikum: 1
A. Source Code 4
B. Output Program 12
C. Pembahasan 17
D. Pembahasan Soal Additional 24
TAUTAN GIT 29
DAFTAR PUSTAKA 30

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 1
Gambar 2. Tampilan Dadu Setelah Di-Roll 2
Gambar 3. Tampilan Roll Dadu Double 3
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Tampilan Awal 12
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Dadu 13
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Dadu Sama 14
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML Tampilan Awal 15
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML Dadu Tidak Sama 16
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML Dadu Sama 17

DAFTAR TABEL

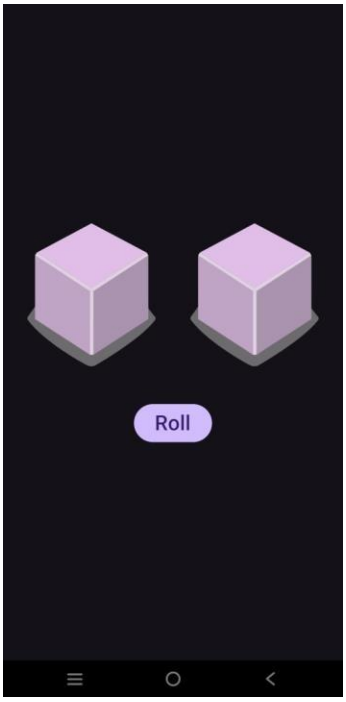
Tabel 1. Source Code Soal 1 MainActivity.kt Compose 4
Tabel 2. Source Code Soal 1 MainActivity.kt XML 8
Tabel 2. Source Code Soal 1 Activity_Main.XML..... 10

SOAL 1

Soal Praktikum:

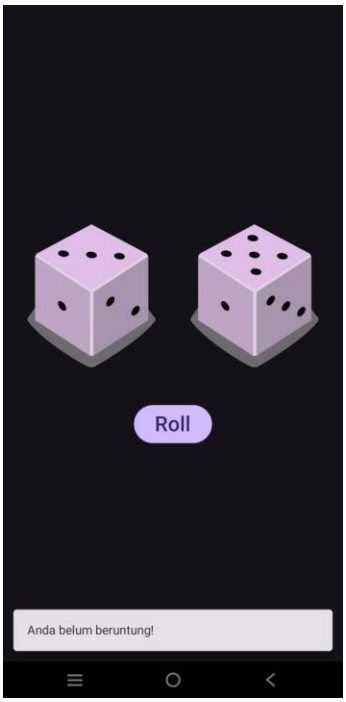
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol "Roll". Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



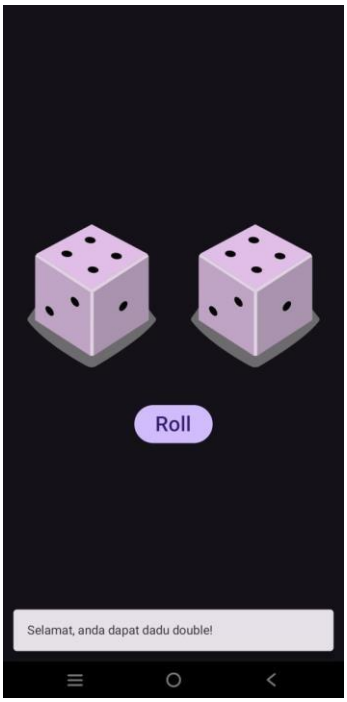
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol "Roll" maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan "Anda belum beruntung!" seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6 Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Soal Additional:
ESSAY

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

** = 2 sumber

* = 1 sumber

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

A. Source Code

MainActivity.kt Compose

Tabel 1. Source Code Soal 1 MainActivity.kt Compose

```
package com.example.compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.compose.ui.theme.ComposeTheme
import
androidx.compose.foundation.layout.wrapContentSize
import androidx.compose.ui.Alignment
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Button
import androidx.compose.ui.res.stringResource
import androidx.compose.foundation.Image
import androidx.compose.ui.res.painterResource
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.Spacer
import androidx.compose.runtime.mutableStateOf
import androidx.compose.ui.unit.dp
import androidx.compose.runtime.remember
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.foundation.layout.size

```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ComposeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    DiceRollerApp(modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }
}

@Preview
@Composable
fun DiceRollerApp(modifier: Modifier = Modifier) {
    DiceWithButtonAndImage(modifier = Modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center)
    )
}

@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    val result1= remember{ mutableStateOf(0) }
    val result2= remember{ mutableStateOf(0) }
    val hasRolled = remember{ mutableStateOf(false)}

    val imageResource1 = when(result1.value){
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    val imageResource2 = when(result2.value){
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
```

```
    }
    Box(modifier = modifier.fillMaxSize()){
        Column (
            modifier = modifier,
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) { Row() {
            Image(
                painter = painterResource(imageResource1),
                contentDescription = result1.value.toString(),
                modifier = Modifier.size(150.dp)
            )
            Image(
                painter = painterResource(imageResource2),
                contentDescription = result2.value.toString(),
                modifier = Modifier.size(150.dp)
            ) }

            Spacer(modifier = Modifier.height(16.dp))
            Button(onClick = {
                result1.value = (1..6).random()
                result2.value = (1..6).random()
                hasRolled.value = true
            }
            ) {
                Text(stringResource(R.string.roll))
            }
        }
            if (hasRolled.value){
                Card(
                    colors = CardDefaults.cardColors(containerColor=Color.Gray),
                    shape = MaterialTheme.shapes.medium,
                    modifier = Modifier
                        .align(Alignment.BottomCenter)
                        .padding(bottom=50.dp)
                        .fillMaxWidth(0.8f)
                ) {
                    Text(
                        text = if (result1.value == result2.value) stringResource(R.string.lucky_message)
                        else stringResource(R.string.unlucky_message),
```

```
                            color = Color.Black,
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(16.dp),
                            textAlign = TextAlign.Center
                        )
                    }
                }
        }
}
```

MainActivity.kt XML

Tabel 2. Source Code Soal 1 MainActivity.kt XML

```
package com.example.dicexml

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
        val rollButton: Button = findViewById(R.id.roll_button)
        rollButton.setOnClickListener{
            rollDice()
        }
    }
    private fun rollDice () {
        val dice1 = Dice(6)
        val dice2 = Dice(6)
        val diceRoll1 = dice1.roll()
        val diceRoll2 = dice2.roll()
        val diceImage1: ImageView = findViewById(R.id.dice_image1)
        val diceImage2: ImageView = findViewById(R.id.dice_image2)
        val resultTextView: TextView = findViewById(R.id.result_text)
        val resultCard : com.google.android.material.card.MaterialCardView = findViewById(R.id.result_card)

        val drawableResource1 = when (diceRoll1) {
```

```
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
        diceImage1.setImageResource(drawableResource1)

        val drawableResource2 = when (diceRoll2) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
        diceImage2.setImageResource(drawableResource2)

        if (diceRoll1 == diceRoll2) {
            resultTextView.text = getString(R.string.lucky_message)
        } else {
            resultTextView.text = getString(R.string.unlucky_message)
        }
        resultCard.visibility =android.view.View.VISIBLE
    }
}

class Dice (val numSides : Int) {
    fun roll() : Int{
        return (1..numSides).random()
    }
}
```

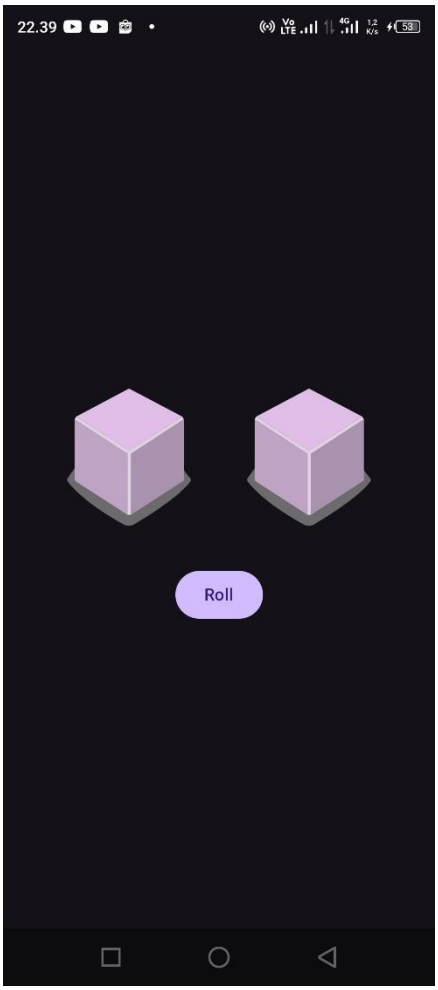
Activity_Main.XML

Tabel 2. Source Code Soal 1 Activity_Main.XML

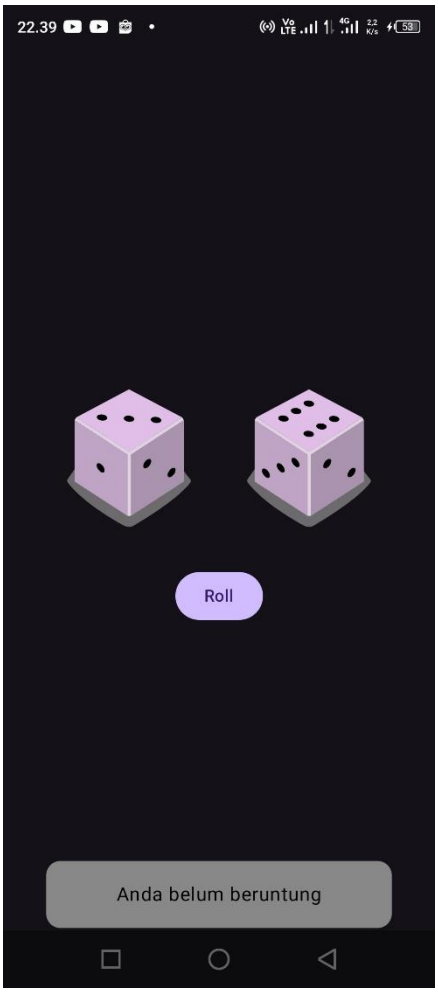
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ImageView
        android:id="@+id/dice_image1"
        android:layout_width="200dp"
        android:layout_height="200dp"
        app:srcCompat="@drawable/dice_0"
        tools:srcCompat="@drawable/dice_1"
app:layout_constraintBottom_toTopOf="@+id/roll_button"
app:layout_constraintEnd_toStartOf="@+id/dice_image2"
app:layout_constraintHorizontal_chainStyle="packed"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
       app:layout_constraintVertical_chainStyle="packed"
        android:layout_marginStart="8dp"/>

    <ImageView
        android:id="@+id/dice_image2"
        app:srcCompat="@drawable/dice_0"
        tools:srcCompat="@drawable/dice_1"
        android:layout_width="200dp"
        android:layout_height="200dp"
app:layout_constraintStart_toEndOf="@+id/dice_image1"
        app:layout_constraintEnd_toEndOf="parent"
     app:layout_constraintTop_toTopOf="@+id/dice_image1"
app:layout_constraintBottom_toBottomOf="@+id/dice_image1
"
        android:layout_marginStart="8dp" />

    <Button
        android:id="@+id/roll_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="50dp"
        android:text="@string/roll"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
```

```
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/dice_image1"
/>

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/result_card"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="20dp"
        android:layout_marginEnd="20dp"
        android:layout_marginBottom="20dp"
        android:visibility="invisible"
        app:cardBackgroundColor="#EEEEEE"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

    <TextView
        android:id="@+id/result_text"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text=""
        android:textColor="@color/black"
        android:padding="20dp"
        android:textAlignment="center" />
    </com.google.android.material.card.MaterialCardView>
</androidx.constraintlayout.widget.ConstraintLayout>
```

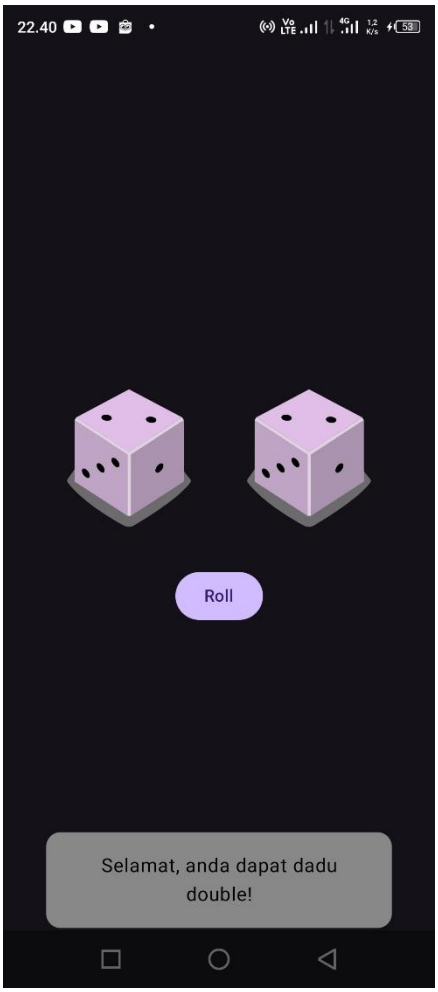
B. Output Program



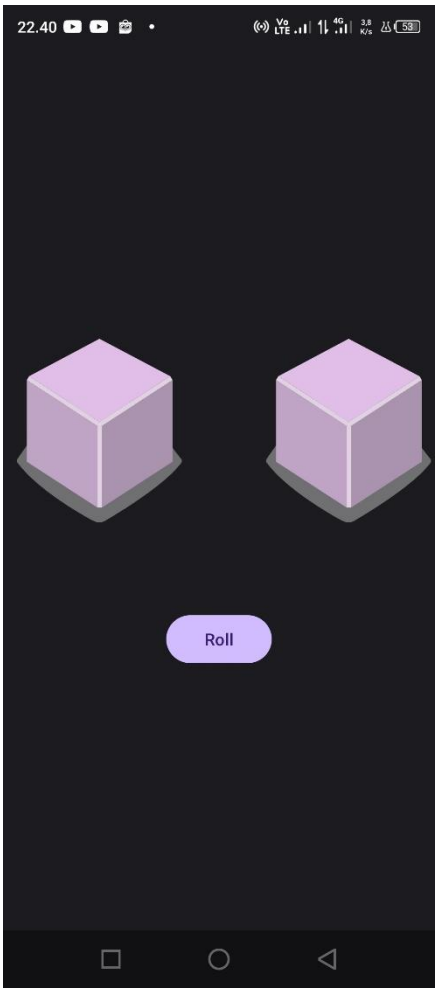
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Tampilan Awal



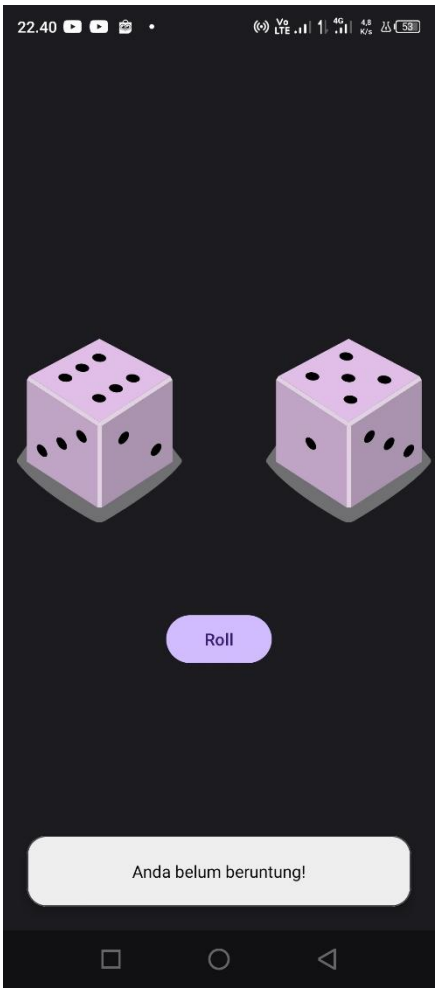
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Dadu



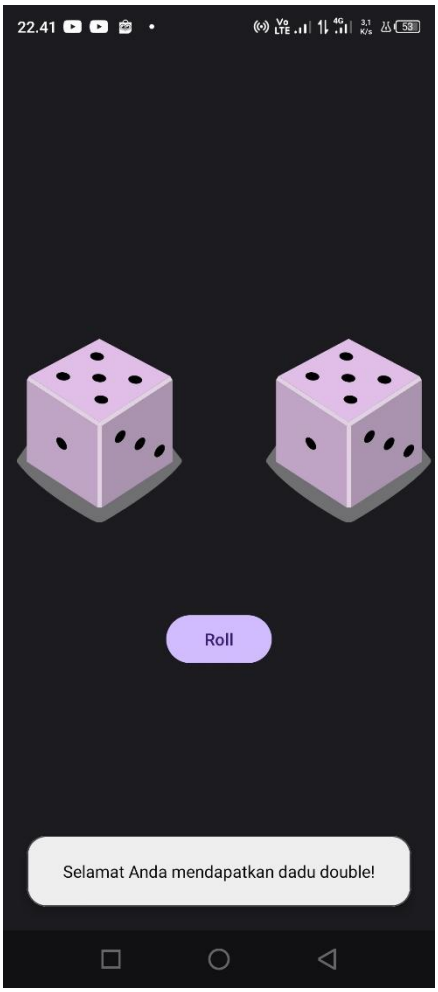
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Dadu Sama



Gambar 7. Screenshot Hasil Jawaban Soal 1 XML Tampilan Awal



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML Dadu Tidak Sama



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML Dadu Sama

C. Pembahasan

MainActivity.kt Compose

Pada baris 1, terdapat `package.com.example.compose` yang berfungsi untuk memudahkan organisir projek dengan mengelompokkan file yang saling berhubungan satu sama lain ke dalam 1 folder package.

Pada baris 2 hingga hingga 36, terdapat semua import yang digunakan pada projek untuk memanggil komponen, library, atau package yang dibutuhkan dalam proses pembuatan aplikasi dice roller, seperti salah satunya adalah material card yang dipanggil agar pada program bisa menggunakan `card()`.

Pada baris 38 hingga 50, terdapat class `MainActivity` yang fungsinya secara dasar adalah sebagai tempat awal berjalannya program. Adapun cara kerja dari class

ini adalah melakukan inheritance pada class `ComponentActivity` yang berasal dari *library*, lalu pada bagian isinya terdapat *function* `onCreate` yang akan dijalankan pertama pada *handphone* ketika aplikasi dijalankan, di mana function tersebut memiliki fungsi untuk memeriksa aktivitas sebelumnya dan menyiapkan tampilan UI sebelum ditampilkan kepada pengguna. Selain itu juga terdapat `setContent()` yang bertugas untuk menampilkan UI, lalu `enableEdgeToEdge` yang memastikan tampilan aplikasi akan memenuhi layer, kemudian ada `ComposeTheme` yang namanya menyesuaikan dengan nama projek di mana isi dari `ComposeTheme` adalah `Scaffold()` yakni komponen layout yang menyediakan struktur standar dari tampilan UI dan pada projek ini diberikan parameter `Modifier.fillMaxSize` dan `.innerPadding` yang memastikan komponen dari aplikasi dapat memenuhi layer handphone serta memastikan tampilan UI tidak tertutup oleh jam ataupun tombol dari handphone. Barulah function dari aplikasi *Dice Roller* di panggil dan diberikan parameter `Modifier.Padding` untuk menerapkan batasan dari `Scaffold` sehingga komponen aplikasi tidak tertutup oleh bagian dari handphone.

Setelahnya barulah pada baris 52 hingga 59 function `DiceRollerApp` yang dipanggil pada `MainActivity()` diinisialisasi, disini function diberikan parameter `modifier` yang mengatur tampilan seperti padding, namun jika nilainya kosong maka akan digunakan nilai `Modifier` kosong tanpa pengaturan tampilan apa-apa. Pada bagian isi dari function `DiceRollerApp`, *function* `DiceWithButtonAndImage` dipanggil dan juga diberikan parameter `modifier.fillMaxSize, .wrapContentSize(Alignment.Center)` untuk mengatur tampilan komponen dari *function* agar memenuhi layar *handphone* dan berada di bagian tengah layar.

Pada baris 61 hingga 65 terdapat inisialisasi function bernama `DiceWithButtonAndImage` yang dipanggil sebelumnya pada function `DiceRollerApp`, funtion ini juga diberikan parameter modifier sesuai standar dari jetpack compose di mana parameter berfungsi agar function menerima pengaturan tampilan dari luar tetapi jika tidak ada maka akan diisi oleh `Modifier` yang nilai defaultnya kosong. Adapun 3 baris isi pertama dari function ini adalah inisialisasi 3

variabel yakni `result1`, `result2`, dan `hasRolled`, di mana ketiganya diatur agar bertipe `val` dan diberi nilai menggunakan `mutableStateOf` untuk memberikan nilai *default* yang dapat berubah disertai dengan `remember` untuk mengingat nilai terakhir dari variabel. Ketiganya diatur menjadi `val` agar nilainya aman dari perubahan tidak terduga yang bisa saja terjadi jika variabelnya diatur sebagai `var` dan bisa dilakukan *reassign*, namun nilainya masih bisa dirubah menggunakan `.value` dikarenakan penggunaan mutablestateof. Adapun ketiganya diberikan nilai default dengan `result1` dan `result2` nilai defaultnya adalah 0, sementara `hasRolled` bernilai awal *false*, masing-masing variabel akan memiliki perannya masing-masing dalam penyusunan logika dari aplikasi *Dice Roller*.

Variabel `result1` dan `result2` berperan dalam penyusunan logika untuk mengatur tampilan dari dadu, di mana logikanya dibuat dengan inisialisasi variabel `imageResource1` dan `imageResource2` yang nantinya akan digunakan sebagai sumber dari tampilan dadu pada baris 67-84. Kedua variabel diatur agar bertipe val dengan nilainya diatur dengan menggunakan penjikaan when di mana pada kasus ini `when` diberikan parameter nilai dari variabel `result1` serta `result2`, yakni apabila nilainya 0 atau belum dilakukan apapun maka akan menampilkan gambar dadu 0, kemudian jika nilainya 1 hingga 5 maka dadunya juga akan menyesuaikan tampilannya menjadi dadu dengan nilai antara 1 sampai dengan 5, dan jika hasilnya selain dari 0 hingga 5 maka tampilan dadu akan menjadi dadu bernilai 6.

Pada baris 85 terdapat `Box()` dengan peran sebagai kontainer bagi komponen-komponen yang dituliskan di dalamnya, `Box()` diberikan parameter `modifier.fillMaxSize` agar memenuhi seluruh bagian layar yang tersedia, dan pada kasus ini `Box()` diisi dengan dua bagian komponen.

Bagian komponen pertama dari `Box()` adalah `column()` yang dituliskan pada baris 86 hingga 111 dengan fungsi untuk menyusun komponen di dalamnya secara vertikal atau dari atas ke bawah, `column()` diberikan parameter modifier kemudian diatur agar komponennya akan terletak tepat di tengah secara horizontal dengan menggunakan `horizontalAlignment= Alignment.Center Horizontally`,

juga secara vertikal menggunakan `verticalArrangement= Arrangement.Center`. Adapun isi komponen dari `Column()` juga terbagi menjadi dua bagian dengan bagian pertama adalah `Row()` yang bertugas menyusun komponen di dalamnya secara horizontal atau bersebelahan, yang pada kasus ini komponen yang disusun bersebelahan adalah dua `Image()`. Masing- masing komponen `Image()` diatur sumber fotonya menggunakan `painterResource()`, kemudian diberikan deskripsi menggunakan `contentDescription` dengan isi deskripsinya akan menyesuaikan dengan nilai dari variabel `result1` dan `result2` yang diubah ke String, dan juga diatur ukurannya menjadi 150dp menggunakan `Modifier.size`.

Bagian komponen kedua dari `Box()` adalah `Button()` yang dituliskan pada baris 103 hingga 107 tepat setelah `Spacer()` yang bertugas memberikan jarak antara komponen pertama dengan komponen kedua. Adapun `Button` diatur agar ketika dia diklik maka akan dilakukan perubahan nilai pada ketiga variabel yang telah diinisialisasi sebelumnya, dengan `result1` dan `result2` dirubah nilainya menggunakan `.value` menjadi angka antara 1 sampai 6 secara acak memakai `random()` dengan kodenya adalah `(1..6).random()`, kemudian juga mengubah nilai dari `hasRolled` menggunakan `.value` menjadi *true* dari yang awalnya bernilai *false*, adapun isi dari `Button` adalah teks dengan sumber tulisan dari teks diambil dari *resource string* bernama `roll` pada `strings.xml`.

Selain kedua komponen tersebut, `Box()` juga diisi dengan logika yang dituliskan pada baris 112 hingga 132 dengan fungsi mengatur apa yang terjadi ketika `hasRolled` berubah nilainya menjadi *true* saat tombol dilakukan klik. Adapun logika disusun menggunakan `if (hasRolled)` yang akan berjalan ketika `hasRolled` bernilai true, dengan logikanya diatur agar ketika itu terjadi akan muncul sebuah `Card()` yang diatur agar berwarna abu dengan shape medium untuk mengatur bagian siku dari `Card()`, serta menerima `modifier .align(Alignment.BottomCenter), .padding(bottom=50.dp), .fillMaxWidth(0.8f)` sehingga akan berposisi pada bagian bawah tengah dari `Box()` dengan jarak dari bagian bawah diatur sebanyak 50dp, dan diatur agar lebarnya 80% dari keseluruhan layar yang tersedia. Adapun isi

dari `Card()` adalah `Text()` yang isi teksnya diatur dengan if agar ketika nilai dari result1 dan result2 sama maka akan menampilkan pesan dari *string resource* bernama `lucky_message` yang berisikan pesan "Selamat Anda mendapat dadu double!", sementara jika tidak sama akan menampilkan pesan dari *string resource* bernama `unlucky_message` yakni "Anda belum beruntung".

MainActivity.kt XML

Pada baris 1, terdapat `package.com.example.dicexml` yang berfungsi untuk memudahkan organisir projek dengan mengelompokkan file yang saling berhubungan satu sama lain ke dalam 1 folder package.

Pada baris 3 hingga hingga 10, terdapat semua import yang digunakan pada projek untuk memanggil komponen, library, atau package yang dibutuhkan dalam proses pembuatan aplikasi dice roller, seperti salah satunya adalah *widget button* yang dipanggil agar pada program bisa membuat tombol.

Sama seperti versi compose, pada versi XML juga terdapat MainActivity yang memiliki tugas dasar yang sama persis namun terdapat perbedaan dalam implementasinya. Salah satu perbedaannya adalah MainActivity versi XML melakukan inheritance dari AppCompatActivity() yang berasal dari library, tetapi versi ini tetap menggunakan onCreate, dan enableEdgeToEdge. Perbedaan lainnya adalah penggunaan setContentView untuk mengatur sumber dari tampilan yakni ActivityMain.xml pada bagian layout, kemudian pengaturan tampilan yang pada versi compose menggunakan modifier pada versi ini dilakukan secara manual pada isi dari setContentView. Selain itu juga terdapat perbedaan di mana tombol langsung dibuat pada MainActivity pada baris 22 dengan menggunakan id dari Button yang telah dibuat pada activity_main.xml, dengan logika dari tombol diatur menggunakan setOnClickListener() agar ketika tombol di klik maka program akan menjalankan function rollDice.

Adapun function rollDice yang dijalankan pada ActivityMain diinisialisasi pada baris 27 hingga 64, di mana di dalamnya pertama-tama dilakukan inisialisasi beberapa variabel bertipe val untuk mencegah nilainya berubah secara tidak sengaja karena

proses reassign. Adapun variabel yang dibuat diantaranya adalah dice1 dan dice2 yang diberi nilai berupa constructor dari Dice dengan nilainya adalah numSides atau yang memiliki peran untuk mengatur jumlah sisi dadu menjadi sebanyak 6, kemudian ada diceRoll1 dan diceRoll2 yang nilainya adalah hasil roll() dari variabel dice1 dan dice2. Varibel berikutnya adalah variabel diceImage1 serta diceImage2 bertipe ImageView di mana keduanya dibuat untuk menampung sumber dari ImageView pada activity_main dengan menggunakan findViewById lalu menuliskan id dari ImageView, juga terdapat resultTextView bertipe TextView dan resultCard bertipe material card yang juga diinisialisasi menggunakan logika yang sama.

Pada baris 37 hingga 55 terdapat inisialisasi dua variabel lagi di mana nilai dari dua variabel tersebut yakni drawableResource1 dan drawableResource2 diatur menggunakan logika penjikaan when di mana nilai dari drawableResource akan mengambil gambar dadu dari resource drawable dengan nilai dari dadunya akan menyesuaikan dengan nilai dari variabel diceRoll1 dan diceRoll2, sehingga jika didapat nilai antara 1 sampai 5 maka gambar dadu yang diambil juga akan menyesuaikan antara gambar dadu bernilai 1 hingga 5, dan jika selain 1-5 maka akan menampilkan gambar dadu bernilai 6. Barulah dari nilai yang didapat oleh drawableResource1 dan drawableResource2, sumber gambar yang ditunjuk oleh variabel diceImage1 serta diceImage2 dirubah menyesuaikan nilai dari drawableResource1 dan drawableResource2.

Pada baris 57 hingga 62 terdapat penjikaan menggunakan if else yang mengatur apabila nilai dari diceRoll1 dan diceRoll2 sama maka resultTextView akan dibuat menampilkan resource string bernama lucky_message yang berisikan pesan “Selamat Anda dapat dadu double!”, namun jika tidak sama maka yang akan ditampilkan oleh resultTextView adalah resource string bernama unlucky_messgae yang berisikan pesan “Anda belum beruntung!”. Barulah pada baris 63 resultCard diatur agar bisa terlihat.

Pada baris 66 hingga 70 terdapat class Dice yang diberi parameter berupa numSides bertipe kan val dengan tipe data Int, inilah yang menjadi sumber constractor Dice yang digunakan pada inisialisasi variabel dice1 dan dice2 sebelumnya. Adapun isi dari class ini adalah function roll() yang juga digunakan pada inisialisasi variabel

sebelumyya, di mana function dirancang mengembalikan nilai bertipe data Int dengan nilainya adalah hasil random dari rentang antara 1 sampai dengan nilai dari numSides.

Activity_main.xml

Pada baris 2 terdapat <android.constraint.layout.widget.ConstraintLayout yang berfungsi sebagai kontainer untuk mengatur tata letak semua komponen di dalamnya. Adapun pada baris 3-4 terdapat baris xml yang mengatur agar program bisa mengakses library luar apabila diperlukan, dan juga memberitahu komputer bahwa file ini berisikan tampilan dari aplikasi android. Kemudian pada baris 5-8 terdapat baris xml yang pertama adalah android:id untuk memberi identitas pada layout yakni main, kemudian layout:width dan layout:height yang berfungsi untuk mengatur lebar dan panjang dari aplikasi yang pada kasus ini diatur agar match_parent atau memaksimalkan luas layar yang tersedia, serta ada context yang memberitahukan komputer jika file xml ini terhubung dengan MainActivity.kt.

Dua komponen pertama yang diatur pada file ini adalah dua komponen Imageview yang pada 5 baris xml pertama dari masing-masing komponennya mengatur id sebagai identitas dari komponen yaitu dice_image1 serta dice_image2, kemudian mengambil sumber tampilan yakni dari resource drawable dice_0, dan mengatur lebar dan panjang dari komponen yang diatur agar bernilai 200dp. Sementara sisa baris xml dari masing-masing komponen merupakan baris yang mengatur constraint atau posisi dari kedua komponen gambar dadu, dice_image1 diminta memposisikan diri ditengah serta berada di bagian pinggir kiri layar, sementara dice_image2 diminta untuk berada di pinggir kanan layar dan diatur agar posisinya mengikuti dice_image1 sehingga jika terjadi pergeseran maka dice_image2 akan secara otomatis mengikuti, meskipun keduanya diminta untuk menempel satu sama lain pada bagian Start_toEnd dan End_tostart, tetapi keduanya tetap diberi jarak sebanyak 8dp pada baris layout_marginstart.

Komponen selanjutnya adalah komponen Button, pada baris pertama ia diberi id sebagai roll_button, kemudian diatur panjang dan lebarnya agar menyesuaikan dengan isi dari tombol, lalu diberikan margin dengan komponen di atasnya sebanyak 50dp, dan diberikan teks “Roll”. Sisa baris xmlnya mengatur constraint atau posisi dari Button di

mana pada kasus ini diatur agar sisi kanan dan kirinya ditarik ke parent atau sisi layar sehingga otomatis akan membuat posisinya berada di tengah, sisi bawah tombol juga diatur untuk ditarik ke bagian bawah layar, sementara sisi atas diatur agar ditarik ke bagian bawah dari komponen dice_image1.

Selanjutnya ada MaterialCardView yang membungkus 1 komponen yakni TextView, adapun untuk MaterialCardView pertama-tama diberikan id, kemudian diatur lebarnya 0dp atau mengikuti margin yang nantinya diberikan, sementara panjangnya diatur mengikuti ukuran isi dari komponen, lalu diberikan margin atau jarak kepada sisi kiri, kanan, dan bawah layar ponsel sebanyak 20dp, dan agar cardview tidak selalu terlihat di aplikasi maka diatur agar invisible sampai ada pemicu yang membuatnya menjadi visible pada MainActivity. Cardview juga diberikan warna latar, diatur tampilan sudutnya dan diberikan constraint untuk mengatur letaknya agar sisi kanan, kiri, dan bawahnya ditarik ke sisi pinggir dan bawah layar. Sementara untuk TextView pertama-tama juga diberikan id, dan diatur panjang dan lebarnya agar mengikuti ukuran dari isinya, adapun teks dibiarkan kosong karena nantinya akan disesuaikan berdasarkan logika pada MainActivity, diatur warna teksnya agar berwarna hitam, diberikan padding sebanyak 20dp agar tidak menempel pada sisi dari cardview, dan diatur agar posisi teksnya berada di tengah-tengah cardview.

D. Pembahasan Soal Additional

1. Imperative adalah paradigma yang berfokus kepada bagaimana menyelesaikan suatu permasalahan, dengan kodenya yang dituliskan detail secara langkah per langkahnya. Adapun penerapannya dalam pengembangan aplikasi android menggunakan Kotlin adalah pada pengembangan UI dari aplikasi menggunakan XML[1]. Sementara declarative adalah paradigma yang berfokus pada apa yang harus dilakukan untuk menyelesaikan suatu permasalahan daripada kepada bagaimana detail langkah per langkahnya untuk mencapai tujuan tersebut[1]. Di mana paradigma declarative menerapkan abstraksi tingkat tinggi yang membuat proses pemrograman menjadi lebih mudah, dan mengurangi pengulangan untuk

mengoptimalkan proses eksekusi[2]. Dengan penerapan declarative pada Kotlin adalah pada perancangan UI menggunakan jetpack compose

Saya sendiri lebih memilih paradigma declarative dikarenakan paradigma imperative itu akan membuat kita banyak melakukan perulangan pada saat menuliskan kode, serta kode yang dituliskan secara detail dapat membuat program menjadi sulit untuk dilakukan maintanance dan debug, terutama jika programnya itu berskala sangat besar, meskipun imperative juga memiliki kelemahan dimana bisa saja abstraksinya terlalu tinggi hingga mungkin saja hanya si penulis kode yang mengerti kodenya, tapi kemudahan dalam maintanance dan debug karena kode yang lebih pendek menurut saya akan sangat membantu sehingga saya memilih paradigma declarative.

2. Penerapan OOP pada Java dan Kotlin memiliki beberapa perbedaan diantaranya adalah pada dasarnya di bahasa Kotlin kita tidak bisa membuat subclass ataupun mewarisi method dari class biasa begitu saja dikarenakan semua class di kotlin telah bersifat final sejak awal, adapun jika kita ingin mengatur agar sebuah class bisa dibuat subclassnya maka kita harus menambahkan open pada nama dari class misalnya open class Dice [3].

   Perbedaan selanjutnya adalah pada Java jika kita tidak menuliskan status dari class misalnya public, private, dan protected yang mengatur dimana saja sebuah class bisa diakses maka secara otomatis class akan bersifat package-private atau bisa terlihat oleh class yang berada dalam 1 package[3]. Sifat ini memiliki sisi positif yakni memudahkan jika memang ingin class bisa diakses oleh class lain di dalam 1 package yang sama tanpa harus diatur menjadi public, tapi terdapat kerentanan dimana package bisa dimasuki oleh class baru dan bisa mengakses class lainnya begitu saja.

   Sementara jika kasus tersebut terjadi pada Kotlin maka sifatnya menjadi public secara default[3]. adapun untuk mengatasi kelemahan dari package-private Java maka Kotlin menggantinya dengan internal yang memungkinkan class bisa dilihat hanya oleh class di modul yang sama sehingga meskipun nanti modul dipanggil oleh

program lain, semua class yang sudah diatur menjadi internal tidak akan bisa diakses.

Perbedaan lainnya adalah pada Java dalam penerapan enkapsulasi, kita perlu untuk membuat setter dan getter agar class lain bisa mengakses nilai dari suatu atribut pada class yang bersifat private, sementara di Kotlin menggunakan konsep property di mana ketika membuat sebuah variabel baik itu bersifat val ataupun var maka setter dan getter dari variabel secara otomatis telah dibuat tanpa harus dituliskan pada class, dan pada implementasinya untuk get kita bisa langsung menunjuk ke variabel secara langsung begitu pula ketika ingin melakukan set. Namun jika kita memerlukan suatu hal yang khusus misalnya perlu mengatur agar nilai tidak minus menggunakan logika if maka kita bisa membuat setter dan getter custom[3].

3. SOLID principles merupakan panduan yang dibuat oleh Robert C. Martin atau dikenal sebagai Uncle Bob, di mana SOLID dirancang untuk membantu perancangan desain dan arsitektur dapat disusun dengan baik, sehingga software menjadi lebih mudah untuk dimengerti, dipelihara, dan diperbarui[4]. SOLID sendiri terdiri atas 5 panduan yakni:
   1. Singe Responsibility Principle (SRP) atau S dari SOLID, arti dari SRP adalah class seharusnya hanya mempunyai satu alasan untuk berubah yang berarti class itu sendiri juga harus hanya mempunyai satu buah tugas atau peran pada sistem. Adapun penerapannya pada kotlin adalah dengan memisahkan class berdasarkan tugasnya masing-masing misalnya class untuk mengambil data dari API misalnya namanya adalah FetchData dengan class yang bertugas menampilkan data tersebut ke UI misalnya DisplayData dibuat menjadi dua class berbeda.
   2. Open/Closed Principle (OCP) atau O dari SOLID, arti dari OCP adalah sistem itu harus bisa dilakukan perubahan seperti penambahan fitur baru tanpa harus merubah apa yang sudah ada, penerapannya pada kotlin bisa dilakukan dengan mengimplementasikan polimorfisme dengan melakukan inheritance dan mengaplikasikan interface, sehingga ketika nanti ingin menambahkan class baru misalnya yang masih berkaitan dengan class yang sudah ada maka pembuatan

class bisa menggunakan class yang sudah ada sebagai parameter tanpa merubah class tersebut.

3. Liskov Subtitution Principle (LSP) atau L dari SOLID, arti dari LSP adalah suatu subclass itu harus bisa menggantikan parentnya tanpa mengakibatkan error pada keseluruhan sistem. Implementasinya pada kotlin adalah dengan memastikan subclass misalnya dari class parent Bird() yang mengharuskan implementasi method fly(), maka penerapan LSV yang baik adalah dengan membuat subclass yang dapat memenuhi kontrak tersebut, jangan memaksakan misalnya jenis burung yang tidak bisa terbang untuk menjadi subclass dari Bird() karna dapat menyebabkan sistem menjadi error atau merubah method fly() menjadi interface tersendiri.
4. Interface Segregation Principle (ISP) atau I dari SOLID, arti dari ISP adalah klien atau class itu tidak boleh dipaksa untuk mengimplementasi sesuatu yang tidak ia butuhkan, contoh implementasinya pada kotlin adalah misal ada sebuah interface bernama Workable() yang berisikan method work() dan eat(), class human mungkin bisa menerima kedua method tersebut, tetapi jika ada class lain yang hanya bisa mengimplementasikan salah satunya seperti robot maka ia tidak boleh dipaksa mengimplementasi method lainnya tersebut, solusinya adalah memisah method work() dan eat() ke dua interface yang berbeda.
5. Dependency Inversion Principle (DIP) atau D dari SOLID, arti dari DIP adalah pada sistem, high-level class atau class yang mengatur alur kerja tanpa perlu mengetahui bagaimana implementasinya tidak boleh bergantung ke low-level atau class yang mengetahui detail bagaimana implementasi yang dimaksud oleh high-level class, keduanya harus bergantung kepada abstraction. Adapun abstraction juga tidak boleh bergantung kepada detail implementasi, melainkan detail implementasi itulah yang bergantung kepada abstraction. Maksud dari konsep ini jika diterapkan pada kotlin adalah dengan mengaplikasikan dependency injection dimana high-level class diatur agar tidak mengimplementasi low-level class secara langsung tetapi sebaiknya melalui parameter, dan low-level class juga mengadaptasi abstraction melalui parameter,

sedangkan abstraction tidak bergantung kepada detail implementasi artinya misal interface itu tidak boleh dibuat khusus untuk satu kasus implementasi, ia harus dibuat agar bisa fleksibel digunakan dalam berbagai kasus implementasi sehingga detail implementasi lah yang nantinhya akan bergantung dengan kontrak yang dibuat oleh interface.

4. Pada Kotlin perbedaan antara class dan data class adalah class biasanya digunakan untuk menyimpan data sekaligus juga untuk menuliskan implementasi berupa method, namun jika pada bagian depan class ditambahkan modifier data atau disebut sebagai data class maka ia menjadi class yang fungsinya hanya untuk menyimpan data saja tanpa harus menuliskan implementasi. Di mana terdapat beberapa keunggulan dari penggunaan data class jika dibandingkan dengan class biasa di mana salah satunya adalah hasil pemeriksaan equals() menggunakan "==" itu akan langsung membandingkan isi datanya jika data disimpan di dalam data class sementara pada class biasa yang dibandingkan adalah alamatnya sehingga nilai yang sama bisa dideteksi tidak equals. Selain itu untuk variabel bertipe val biasanya akan menggunakan copy() jika ingin melakukan perubahan pada nilainhya namun jika pada class biasa copy() harus dibuat secara manual sementara jika pada data class maka Kotlin akan secara otomatis membuatkan copy() sehingga bisa langsung digunakan.

TAUTAN GIT

https://github.com/HamkaArifani/Praktikum-Pemrograman-Mobile

DAFTAR PUSTAKA

[1] M. S. Saeed, "Mobile Application Development using Kotlin," Tesis Sarjana, Faculty of Information Technology and Communication Sciences, Tampere University, Tampere, 2024. [Daring]. Tersedia: https://www.doria.fi/bitstream/handle/10024/190688/saeed_muhammad%20suleman.pdf?sequence=3&isAllowed=y

[2] J. M. Simão, C. A. Tacla, P. C. Stadzisz, dan R. F. Banaszewski, "Notification Oriented Paradigm (NOP) and Imperative Paradigm: A Comparative Study," *Journal of Software Engineering and Applications*, vol. 5, no. 6, hal. 402-416, Jun. 2012, doi: 10.4236/jsea.2012.56047.

[3] S. Isakova, R. Elizarov, S. Aigner, dan D. Jemerov, *Kotlin in Action*, edisi ke-2, MEAP v09. Shelter Island, NY, USA: Manning Publications, 2023.

[4] N. S. P. K. Yadati, "Solid Principles in Android Development with Kotlin," *Journal of Scientific and Engineering Research*, vol. 6, no. 2, hal. 282-286, 2019.